Python Generators

Generators

Generators are functions that can pause and resume their execution.

When a generator function is called, it returns a *generator object*, which is an iterator.

The code inside the function is not executed yet, it is only compiled. The function only executes when you iterate over the generator.

A simple generator function:

```
def my_generator():
  yield 1
  yield 2
  yield 3

for value in my_generator():
  print(value)
```

OUTPUT:

```
1
2
3
```

Generators allow you to iterate over data without storing the entire dataset in memory.

Instead of using return, generators use the yield keyword.

The yield Keyword

The `yield` keyword is what makes a function a generator.

When `yield` is encountered, the function's state is saved, and the value is returned. The next time the generator is called, it continues from where it left off.

Generator that yields numbers:

```
def count_up_to(n):
  count = 1
  while count <= n:
    yield count
    count += 1

for num in count_up_to(5):
  print(num)
```

OUTPUT:

```
1
2
3
4
5
```

Unlike `return`, which terminates the function, `yield` pauses it and can be called multiple times.

Generators Saves Memory

Generators are memory-efficient because they generate values on-the-fly instead of storing everything in memory.

For large datasets, generators save memory:

Generator for large sequences:

```
def large_sequence(n):
  for i in range(n):
    yield i

# This doesn't create a million numbers in memory
gen = large_sequence(1000000)
print(next(gen))
print(next(gen))
print(next(gen))
```

OUTPUT:

```
0
1
2
```

Using next() with Generators

You can manually iterate through a generator using the `next()` function:

Example

```
def simple_gen():
  yield "Emil"
  yield "Tobias"
  yield "Linus"

gen = simple_gen()
print(next(gen))
print(next(gen))
print(next(gen))
```

OUTPUT:

```
Emil
Tobias
Linus
```

When there are no more values to yield, the generator raises a `StopIteration` exception:

Example

```
def simple_gen():
  yield 1
  yield 2

gen = simple_gen()
print(next(gen))
print(next(gen))
print(next(gen)) # This will raise StopIteration
```

OUTPUT:

```
1
2
---------------------------------------------------------------------------
StopIteration                             Traceback (most recent call last)
<ipython-input-5-71cada57984a> in <module>
      6 print(next(gen))
      7 print(next(gen))
----> 8 print(next(gen))

StopIteration:
```

Generator Expressions

Similar to list comprehensions, you can create generators using generator expressions with parentheses instead of square brackets:

List comprehension vs generator expression:

```
# List comprehension - creates a list
list_comp = [x * x for x in range(5)]
print(list_comp)
```

OUTPUT: `[0, 1, 4, 9, 16]`

```
# Generator expression - creates a generator
gen_exp = (x * x for x in range(5))
print(gen_exp)
print(list(gen_exp))
```

OUTPUT:

```
<generator object <genexpr> at 0x000001D5F15B0040>
[0, 1, 4, 9, 16]
```

```
gen_exp = (x * x for x in range(5))
print(gen_exp)
print(tuple(gen_exp))
```

OUTPUT:

```
<generator object <genexpr> at 0x000001D5F15B0040>
(0, 1, 4, 9, 16)
```

Using a generator expression with sum:

```
# Calculate sum of squares without creating a list
total = sum(x * x for x in range(10))
print(total)
```

Output: 285